Chapter 7
Software Configuration Management

Contents:

Introduction:

Software Configuration Management(SCM) is a process to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle.

The primary goal is to increase productivity with minimal mistakes.

Software configuration management is referred to as source control, change management, and version control.

Software configuration management systems are commonly used in software development groups in which several developers are concurrently working on a common set of files.

If two developers change the same file, that file might be overwritten and critical code changes lost.

Software configuration management systems are designed to avoid this inherent problem with sharing files in a multiuser environment.

Any software configuration management system creates a central repository to facilitate file sharing.

Each file to be shared must be added to the central repository to create the first version of the file. After a file is part of the central repository, users can access and update it, creating new versions.

Why do we need Configuration management?

- There are multiple people working on software which is continually updating
- It may be a case where multiple version, branches, authors are involved in a software config project, and the team is geographically distributed and works concurrently
- Changes in user requirement, policy, budget, schedule need to be accommodated.
- Helps to develop coordination among stakeholders
- SCM process is also beneficial to control the costs involved in making changes to a system

The main advantages of SCM are:

1. Improved productivity and efficiency by reducing the time and effort required to manage software changes.
2. Reduced risk of errors and defects by ensuring that all changes are properly tested and validated.
3. Increased collaboration and communication among team members by providing a central repository for software artifacts.
4. Improved quality and stability of software systems by ensuring that all changes are properly controlled and managed.

The main disadvantages of SCM are:

1. Increased complexity and overhead, particularly in large software systems.
2. Difficulty in managing dependencies and ensuring that all changes are properly integrated.
3. Potential for conflicts and delays, particularly in large development teams with multiple contributors.

Element of SCM:

System Configuration Management (SCM) encompasses several key elements to effectively manage the configuration of systems throughout their lifecycle. Here are the main elements of SCM:

1. **Configuration Identification**: This involves identifying and defining the configuration items (CIs) that make up the system, including software, hardware, documentation, and other related components.

   Each CI is uniquely identified and documented to facilitate tracking and management.

2. **Change Management**: Change management processes govern how changes to the system's configuration are proposed, evaluated, approved, and implemented.

   This includes assessing the impact of proposed changes, managing change requests, and ensuring that changes are properly documented and communicated.

3. **Version Control**: Version control, also known as revision control or source control, is essential for managing different versions of configuration items.

It involves tracking changes, managing concurrent edits, and enabling collaboration among team members while maintaining a history of revisions.

4. **Configuration Control**: Configuration control processes ensure that the system's configuration remains consistent and aligned with its requirements, design, and standards.

   This involves enforcing policies and procedures for making changes, controlling access to configuration items, and maintaining integrity throughout the configuration lifecycle.

5. **Configuration Status Accounting**: Configuration status accounting involves tracking and reporting the status of configuration items throughout their lifecycle.

   This includes recording changes, versions, relationships between items, and other relevant information to provide visibility and traceability.

6. **Configuration Auditing and Verification**: Regular audits and verification activities ensure that the actual configuration of the system matches its intended configuration.

This involves comparing configurations against baselines, conducting inspections, and verifying compliance with standards and requirements.

7. **Documentation and Baselines**: Documentation is essential for capturing and communicating information about the system's configuration, including specifications, designs, and change history.

   Baselines represent stable configurations at specific points in time and serve as references for managing changes.

   A baseline refers to a predefined and approved version of a configuration item or a set of configuration items at a specific point in time.

   Baselines serve as reference points or snapshots of the system's configuration state and are used for various purposes throughout the system's lifecycle.

Baseline:

A baseline refers to a predefined and approved version of a configuration item or a set of configuration items at a specific point in time.

Baselines serve as reference points or snapshots of the system's configuration state and are used for various purposes throughout the system's lifecycle.

Baselines typically represent:

1. **Controlled Configuration**: Baselines help establish control over changes to the system by defining a set of configuration items and their versions. Once a baseline is established, any changes made to the configuration must go through formal change management processes.
2. **Comparative Reference**: Baselines provide a basis for comparison when evaluating changes or troubleshooting issues. By comparing the current configuration to a baseline, stakeholders can identify differences, track modifications, and assess the impact of changes.

3. **Auditing and Compliance**: Baselines serve as checkpoints for auditing and compliance purposes. They document the configuration state at specific milestones, helping ensure that the system meets regulatory requirements, industry standards, and organizational policies.
4. **Reproducibility**: Baselines facilitate reproducibility by capturing the exact state of the system at a particular point in time. This is valuable for recreating environments, conducting regression testing, and addressing issues that may arise in specific configurations.

SCM Repository:

Paper-based vs. Automated Repositories

- Problems with paper-based repositories (i.e., file cabinet containing folders)
  - Finding a configuration item when it was needed was often difficult
  - Determining which items were changed, when and by whom was often challenging

- Constructing a new version of an existing program was time consuming and error prone
- Describing detailed or complex relationships between configuration items was virtually impossible

- Today's automated SCM repository
  - It is a set of mechanisms and data structures that allow a software team to manage change in an effective manner
  - It acts as the center for both accumulation and storage of software engineering information
  - Software engineers use tools integrated with the repository to interact with it

A repository is a centralized location where all versions of source code, documentation, configuration files, and other artifacts related to a software project are stored and managed.

The SCM repository serves as a single source of truth for the project's assets and facilitates collaboration, version control, and traceability among team members.

Here's a more detailed explanation:

1. **Centralized Storage**: The repository provides a centralized storage location for all project assets, eliminating the need for scattered file systems or individual developers' machines to store and manage files.

   This ensures that all team members have access to the latest versions of project artifacts.

2. **Version Control**: One of the primary functions of an SCM repository is version control.

   It keeps track of changes made to files over time, allowing developers to access previous versions, compare changes, and revert to earlier states if necessary.

   Version control helps maintain the integrity and consistency of the project's codebase and other assets.

3. **Collaboration**: The repository facilitates collaboration among team members by providing a shared workspace where developers can work on the same files simultaneously.

Version control mechanisms ensure that changes made by different developers are properly managed and merged, preventing conflicts and ensuring that everyone is working with the latest code.

4. **History and Auditability**: The repository maintains a comprehensive history of changes made to project files, including who made the changes, when they were made, and what specific modifications were implemented.

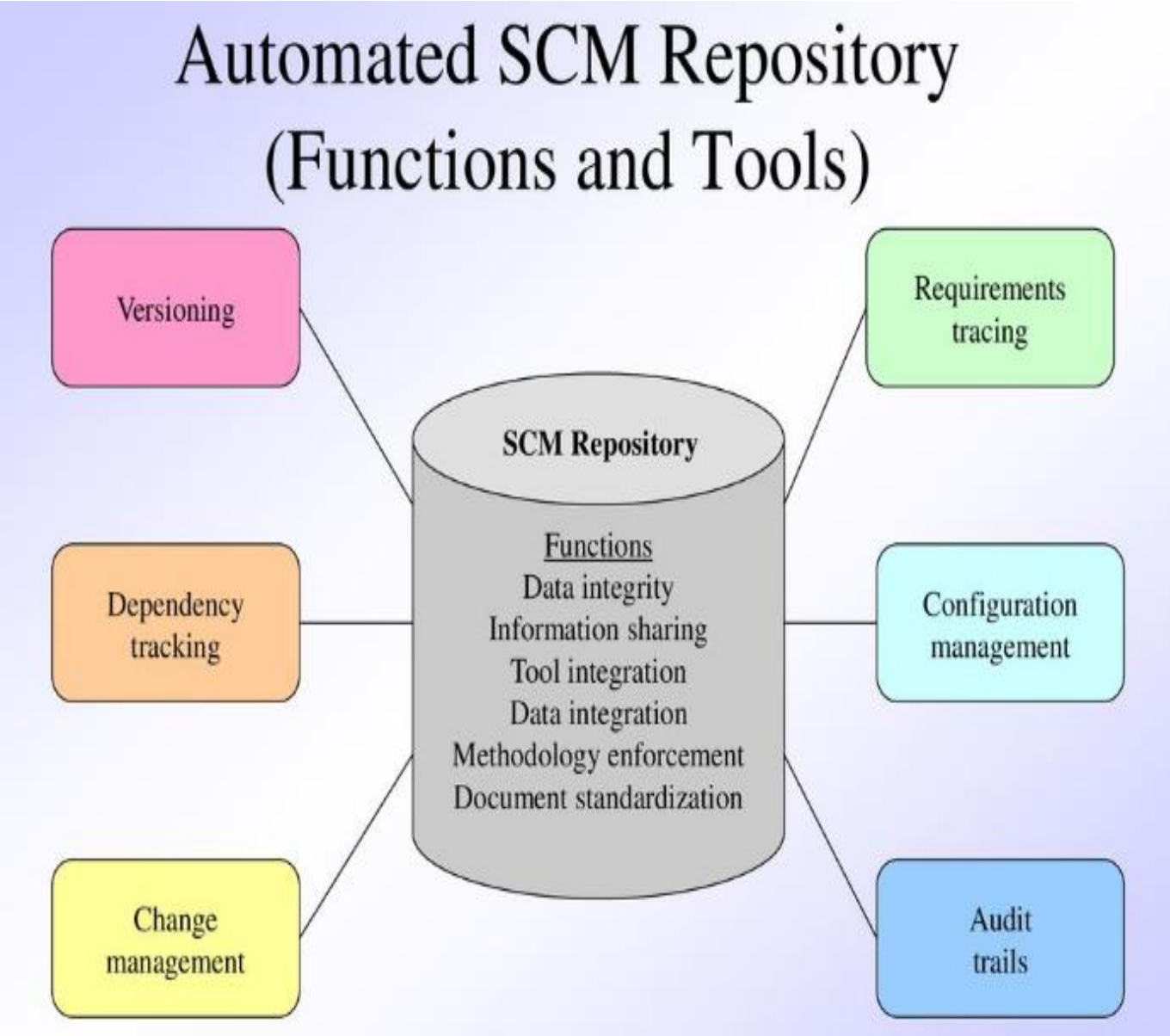
   This audit trail is valuable for tracking the evolution of the project, identifying the root causes of issues, and ensuring compliance with regulatory requirements.

5. **Branching and Merging**: SCM repositories support branching and merging, allowing developers to create separate lines of development (branches) for implementing new features or addressing specific issues without affecting the main codebase.

   Branches can be merged back into the main branch once changes are tested and approved, enabling a structured and controlled development process.

6. **Access Control**: Repositories often include access control mechanisms to restrict access to certain files or directories based on user roles and permissions.

This ensures that sensitive or proprietary information is protected and that only authorized individuals can make changes to critical components of the project.



Functions of an SCM Repository:

1. **Data Integrity**: The repository ensures the integrity of data by validating entries, ensuring consistency, and cascading modifications throughout related items to maintain coherence.
2. **Information Sharing**: It facilitates information sharing among developers and tools by providing a centralized location for storing and accessing project artifacts. It also manages and controls multi-user access to prevent unauthorized modifications.
3. **Tool Integration**: The repository establishes a data model that can be accessed by various software engineering tools, enabling seamless integration with version control, issue tracking, build automation, and other development tools. It controls access to the data to maintain security and consistency.
4. **Data Integration**: It allows various SCM tasks to be performed on one or more Configuration Item (CI), enabling developers to manage and manipulate project artifacts efficiently.
5. **Methodology Enforcement**: The repository defines an entity-relationship model that reflects a specific process model for software engineering.

It enforces methodology guidelines and standards by providing a structured framework for organizing and managing project artifacts.

6. **Document Standardization**: It defines objects in the repository to ensure a standard approach for the creation of software engineering documents. This helps maintain consistency and coherence across project documentation.

Toolset Used on a Repository:

1. **Versioning**: Allows users to save and retrieve all repository objects based on version numbers, facilitating version control and tracking changes over time.
2. **Dependency Tracking and Change Management**: Tracks and manages dependencies between repository objects and enables users to respond to changes in the state and relationships of these objects.
3. **Requirements Tracing**: Supports both forward tracing, which tracks the design and construction components resulting from specific requirements specifications, and backward tracing, which identifies the requirements that generated specific work products.

4. **Configuration Management**: Tracks a series of configurations representing specific project milestones or production releases, ensuring that the project's configuration remains consistent and reproducible.
5. **Audit Trails**: Establishes records of when, why, and by whom changes are made in the repository, providing a comprehensive audit trail for accountability and compliance purposes.

Versioning and Version Control:

Versioning and version control are closely related concepts within the context of software development and configuration management. Here's an explanation of each:

1. **Versioning**:

   Versioning refers to the practice of assigning unique identifiers or labels to different iterations or releases of software or other digital assets.

These identifiers typically take the form of version numbers, which may include numeric sequences (e.g., 1.0, 1.1, 2.0) or alphanumeric codes (e.g., v1.0, v1.1-beta, v2.0-alpha).

The purpose of versioning is to differentiate between different versions of a software product or project, typically indicating changes, updates, or improvements made over time.

Version numbers often convey information about the significance of changes, with major version increments signifying substantial updates and minor version increments indicating smaller changes or bug fixes.

Versioning is essential for tracking the evolution of software, managing dependencies between different versions, and facilitating communication among developers, users, and stakeholders.

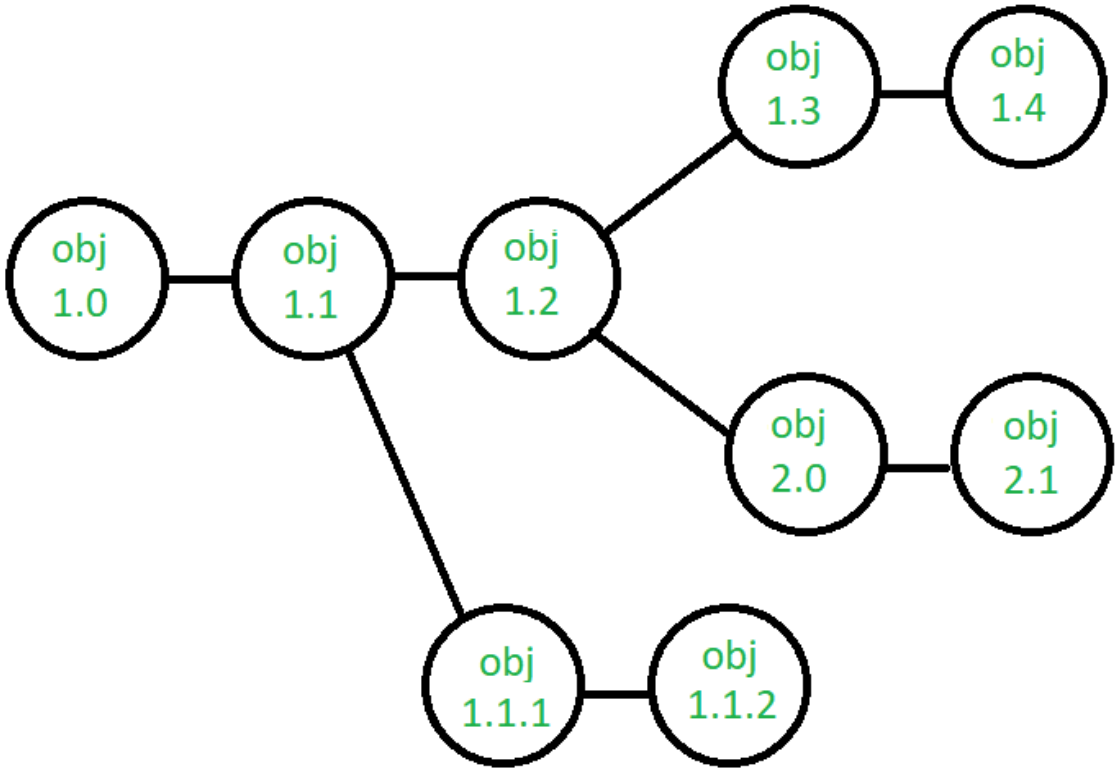
2. **Version Control**:

Version control, also known as revision control or source control, is a system or process for managing changes to files, documents, or other artifacts over time. It enables developers to track modifications, collaborate on projects, and maintain a history of changes made to software code, documentation, configuration files, and other assets.

Key features of version control systems include:

- **Repository**: A centralized or distributed database or storage system where all versions of files and associated metadata are stored.
- **Check-in/Check-out**: Mechanisms for locking and unlocking files to prevent simultaneous editing by multiple users and ensure data consistency.
- **Revision History**: A record of all changes made to files, including details such as the author of each change, the timestamp, and a description of the modifications.
- **Branching and Merging**: Facilities for creating separate branches of development (e.g., feature branches, release branches) and merging changes between branches to integrate new features or bug fixes.
- **Conflict Resolution**: Tools and processes for resolving conflicts that arise when multiple users attempt to modify the same file concurrently.

Version control systems enable developers to work collaboratively, experiment with changes without fear of losing previous work, track the evolution of projects, and manage complex software development workflows effectively.



Example:

Imagine you're an author writing a book. You start with the first draft, which you label as Version 1.0. After completing this initial draft, you share it with a few friends for feedback. Based on their suggestions, you make some revisions and improvements to the manuscript.

Now, your revised manuscript represents Version 1.1. You incorporate feedback, fix some typos, and clarify certain sections. You may decide to share this updated version with a larger group of beta readers to gather more feedback.

As you continue to refine your book, you reach a significant milestone where you believe it's ready for publication. This becomes Version 2.0—a major update that includes significant changes, perhaps restructuring chapters, adding new content, or revising the overall tone and style.

SCM Process:

1. Configuration Identification
2. Baselines
3. Change Control
4. Configuration Status Accounting
5. Configuration Audits and Review

1. **Configuration Identification**:

This process involves identifying and defining the configuration items (CIs) that make up a system or project. CIs can include software code, documentation, hardware components, configuration files, and other related items. The goal is to establish a comprehensive inventory of all the elements that contribute to the system's functionality and behavior.

Example:

Imagine you're working on a software project to develop a web application. Configuration items for this project may include:

- Source code files (HTML, CSS, JavaScript, etc.).
- Database schema and scripts.
- Configuration files (e.g., for server settings, application parameters).
- Documentation (user manuals, technical specifications).
- Third-party libraries or dependencies.

By identifying these configuration items, you create a baseline understanding of what constitutes the project's deliverables and components.

2. **Baselines**:

Baselines represent predefined and approved versions of configuration items at specific points in time. They serve as reference points for tracking changes and managing the configuration of the system throughout its lifecycle. Baselines capture stable configurations that have been tested, verified, and approved for use in development, testing, or production environments.

Example:

In our web application project, you might establish a baseline after completing the initial development phase and successfully testing the core functionality. This baseline would include:

- The source code files for the application.
- The database schema and initial data.
- Configuration files with settings optimized for the development environment.

This baseline serves as a foundation for further development, testing, and deployment activities.

3. **Change Control**:

Change control involves managing and controlling changes to the configuration items within the system. It includes processes for requesting, evaluating, approving, implementing, and documenting changes to ensure that they are made in a controlled and coordinated manner, minimizing risks and disruptions.

Example:

Suppose a stakeholder requests a new feature to be added to the web application. The change control process would involve:

- Submitting a change request specifying the requested feature and its requirements.
- Evaluating the impact of the change on the project's schedule, budget, and resources.
- Obtaining approval from the appropriate stakeholders or change control board.
- Implementing the approved change, which may involve modifying source code, updating documentation, or reconfiguring the system.
- Documenting the changes made and updating relevant documentation or baselines.

4. **Configuration Status Accounting**:

Configuration status accounting involves tracking and reporting the status of configuration items throughout their lifecycle. It includes recording changes, versions, relationships between items, and other relevant information to provide visibility and traceability.

Example:

As development progresses on the web application, configuration status accounting would involve:

- Recording changes made to source code files, including who made the changes and when.
- Tracking versions of database schema changes and documenting any modifications to data structures or relationships.
- Updating configuration files to reflect changes in server settings or application parameters.
- Maintaining documentation to reflect the current state of the system and its components.

This information helps stakeholders understand the evolution of the project and ensures that the configuration remains consistent and up-to-date.

5. **Configuration Audits and Review**:

Configuration audits and reviews involve evaluating the configuration of the system to ensure compliance with requirements, standards, and best practices. Audits may be conducted periodically or in response to specific events to verify that the system's configuration aligns with expectations and to identify any discrepancies or issues.

Example:

Before releasing the web application to production, you might conduct a configuration audit to:

- Review the source code for adherence to coding standards, security practices, and performance guidelines.
- Validate the database schema against design specifications and data integrity requirements.
- Verify that configuration files are correctly configured for the target environment and meet security and compliance requirements.
- Ensure that documentation is comprehensive, up-to-date, and accurately reflects the system's configuration and functionality.

Configuration audits help identify areas for improvement, mitigate risks, and ensure the overall quality and integrity of the system configuration.

Version Control Software Tool:

Git:

What is Git?

Git is an open-source distributed version control system.

It is designed to handle minor to major projects with high speed and efficiency.

It is developed to co-ordinate the work among the developers.

The version control allows us to track and work together with our team members at the same workspace.

Git is foundation of many services like **GitHub** and **GitLab**, but we can use Git without using any other Git services. Git can be used **privately** and **publicly**.

Git was created by **Linus Torvalds** in **2005** to develop Linux Kernel. It is also used as an important distributed version-control tool for **the DevOps**.

Git is easy to learn, and has fast performance. It is superior to other SCM tools like Subversion, CVS, Perforce, and ClearCase.

Features of Git

Some remarkable features of Git are as follows:

- **Open Source**

  Git is an **open-source tool**. It is released under the **GPL** (General Public License) license.

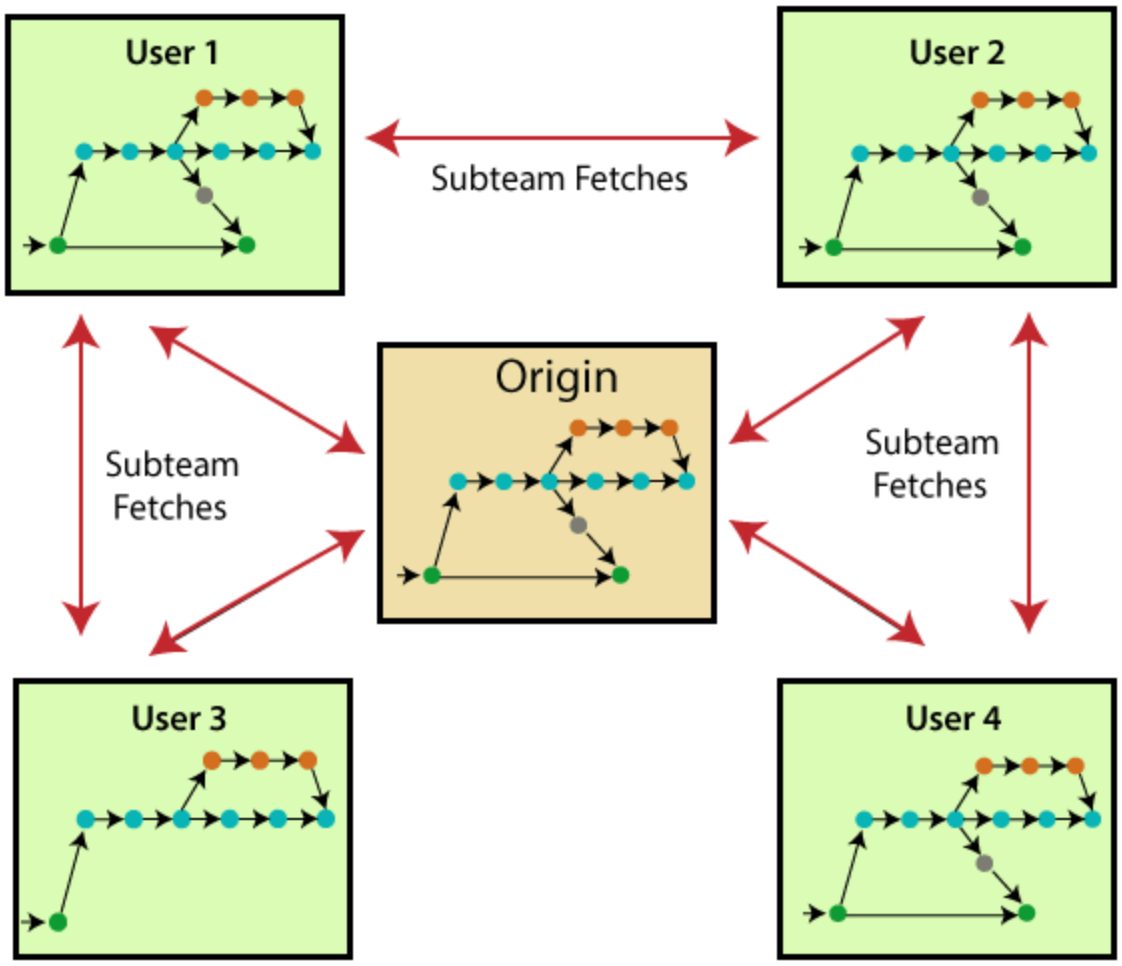
- **Scalable**
  Git is **scalable**, which means when the number of users increases, the Git can easily handle such situations.
- **Distributed**
  One of Git's great features is that it is **distributed**.

  Distributed means that instead of switching the project to another machine, we can create a "clone" of the entire repository.

  Also, instead of just having one central repository that you send changes to, every user has their own repository that contains the entire commit history of the project.

  We do not need to connect to the remote repository; the change is just stored on our local repository.

  If necessary, we can push these changes to a remote repository.



- **Security**

  Git is secure.

  It uses the **SHA1 (Secure Hash Function)** to name and identify objects within its repository.

  Files and commits are checked and retrieved by its checksum at the time of checkout.

It stores its history in such a way that the ID of particular commits depends upon the complete development history leading up to that commit.

Once it is published, one cannot make changes to its old version.

- **Speed**

  Git is very **fast**, so it can complete all the tasks in a while.

  Most of the git operations are done on the local repository, so it provides a **huge speed**.

- **Branching and Merging**

  Branching and merging are the **great feature**s of Git, which makes it different from the other SCM tools.

  Git allows the **creation of multiple branches** without affecting each other.

  We can perform tasks like **creation**, **deletion**, and **merging** on branches, and these tasks take a few seconds only.

- **Data Assurance**

  The Git data model ensures the **cryptographic integrity** of every unit of our project.

  It provides a **unique commit ID** to every commit through a **SHA algorithm**.

  We can **retrieve** and **update** the commit by commit ID. Most of the centralized version control systems do not provide such integrity by default.

- **Staging Area**

  The **Staging area** is also a **unique functionality** of Git.

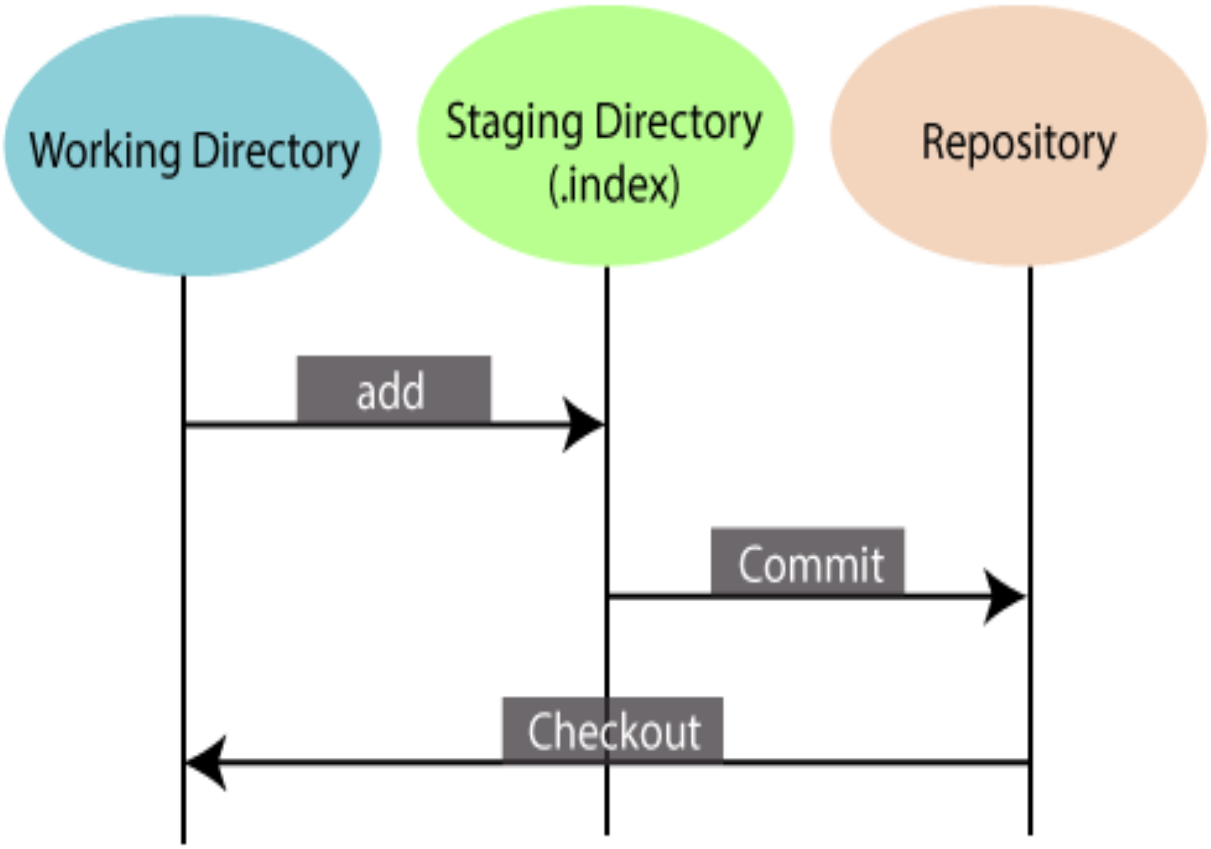
  It can be considered as a **preview of our next commit**, moreover, an **intermediate area** where commits can be formatted and reviewed before completion.

  When you make a commit, Git takes changes that are in the staging area and make them as a new commit.

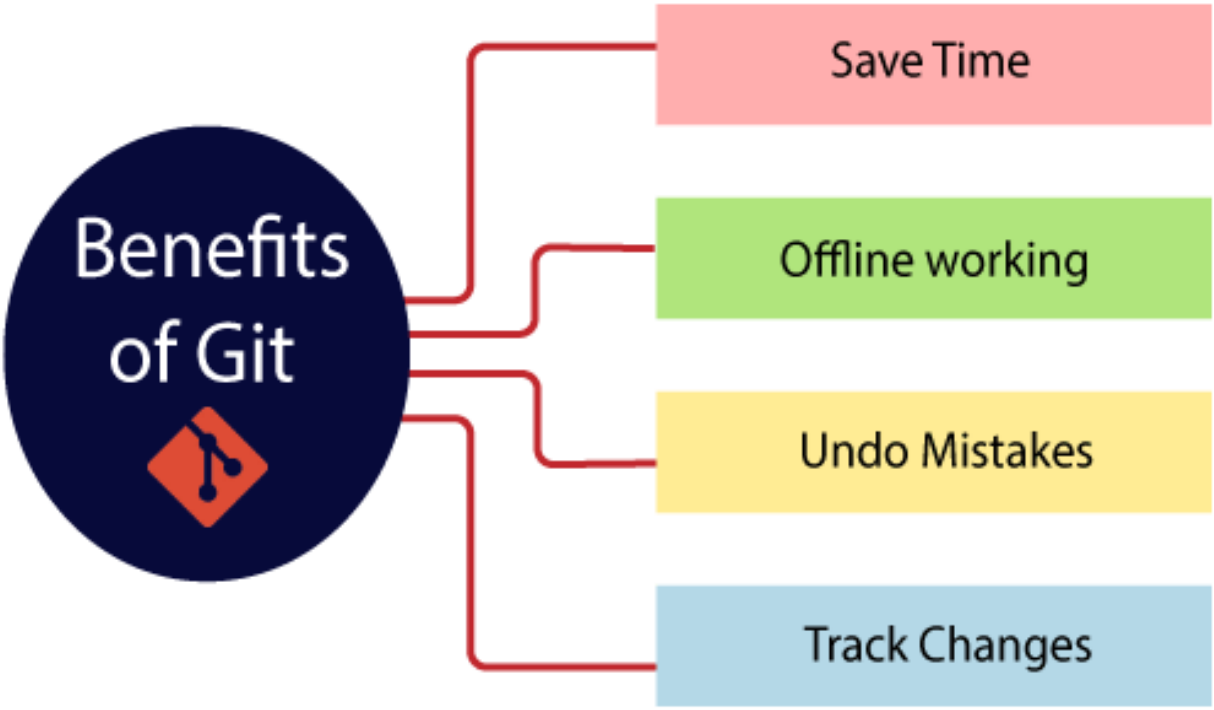
  We are allowed to add and remove changes from the staging area.

  The staging area can be considered as a place where Git stores the changes.

Although, Git doesn't have a dedicated staging directory where it can store some objects representing file changes (blobs). Instead of this, it uses a file called index.



Benefits of Git



Concurrent Versions System (CVS) :

Concurrent versions System is functional version control system which is developed by Dick Grune as series of shell scripts.

This helps the teams to be connected to the changes that are measured into repository when working on software.

This tool was used as the version control system for long for time. It is reliable software tool but with new challenges other alternatives made it used as limited.

Following are some features of CVS :

- It is one of the reliable Version Control System
- It does not allows commit with errors to made.
- Scripts are written in RCS.
- User can only store files into repository.

Advantages :

- CVS is one of the reliable version control software.
- Changes are committed with full change.

Disadvantages :

- CVS changes are time consuming.
- CVS do not commit if there is error into the commit.

Subversion (SVN) :

Subversion is an open-source highly functional version control system which is developed by CollabNet Inc and then later was taken by Apache Software Foundation.

This system has a centralized control by a server.

This server is responsible for the storage of repository, and through the server, this repository is then distributed into three areas called Branch, Area, and Truck. This makes the SVN to be different.

Following are some features of SVN :

- It binds with a diverse number of programming languages.
- When multiple people access the same file it is locked called File Locking.
- Directives and editing are versioned.
- Binary files are handled with care.

Advantages :

- SVN is highly configurable.
- Latest changes are committed and observed only.

Disadvantages :

- SVN central repository makes commit changes time consuming.
- It is difficult to learn SVN for the first timers.